

ECB CBC CTR Comparison

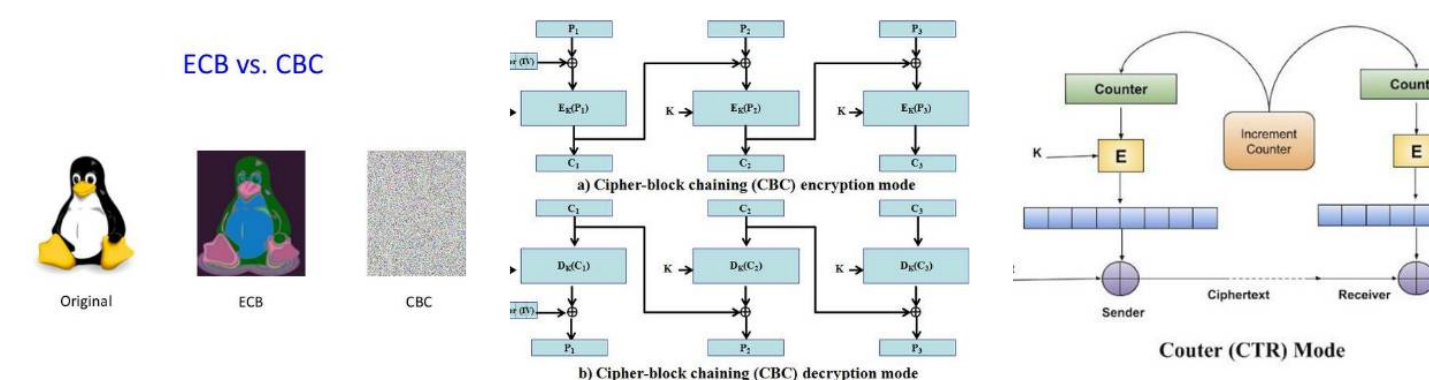


1. Compare **ECB** with **CBC** and **CTR** modes in terms of: Confidentiality, Error propagation, Parallelization, and Security against pattern leakage

10 Marks

Here's a **clean, exam-ready comparison** of ECB, CBC, and CTR modes:

🔑 ECB vs CBC vs CTR Modes



Feature	ECB (Electronic Codebook)	CBC (Cipher Block Chaining)	CTR (Counter Mode)
Confidentiality	❌ Weak – identical plaintext blocks → identical ciphertext	✅ Good – chaining hides patterns	✅ Strong – behaves like stream cipher
Pattern Leakage	❌ High – patterns visible (not secure)	⚠️ Reduced – patterns hidden due to chaining	✅ No pattern leakage
Error Propagation	♦ One block error affects only that block	♦ Error affects current + next block	♦ Error affects only corresponding bit/block
Parallelization	❌ Not parallelizable	❌ Encryption sequential (Decryption parallelizable)	✅ Fully parallelizable (both enc & dec)
Security Level	❌ Least secure	✅ Moderate security	✅ High security

🧠 Key Points to Memorize

- **ECB:**
Same input → same output → **pattern leakage** → **insecure**
- **CBC:**
Uses **IV + chaining**, hides patterns but **not parallel (encryption)**

- **CTR:**
Uses **counter + keystream**, fastest & most secure among three
→ **parallel + no error propagation**

One-line Summary (for exams)

- **ECB:** Simple but insecure (pattern leakage)
- **CBC:** Secure but slower (sequential dependency)
- **CTR:** Most efficient and secure (parallel + no patterns)

If you want, I can give you a **perfect 10-mark answer format (paragraph style)** to directly write in exams.



2. A web application encrypts session tokens deterministically.
Explain how a Chosen Plaintext Attack adversary can distinguish between two chosen inputs.
--15 marks

Give proper descriptive answer

Here is a **proper descriptive, exam-ready answer (15 marks)**:

Chosen Plaintext Attack on Deterministic Encryption

A web application encrypts session tokens **deterministically**, meaning that the same plaintext input always produces the same ciphertext output. This property makes the system vulnerable to a **Chosen Plaintext Attack (CPA)**.

Key Idea of the Attack

In a CPA, the adversary is allowed to:

- Choose plaintext inputs of their choice
- Obtain corresponding ciphertexts from the encryption system

The goal is to **distinguish between two chosen plaintexts** by analyzing their ciphertexts.

Attack Process

1. Adversary selects two plaintexts

Let the attacker choose two possible session tokens:

- P_0 (e.g., "user=guest")
- P_1 (e.g., "user=admin")

2. Challenge Phase

The attacker sends both P_0 and P_1 to the system.

The system randomly selects one of them (say P_b , where $b \in \{0, 1\}$) and returns:

$$C = E(P_b)$$

3. Exploiting Determinism

Since encryption is deterministic:

- $E(P_0)$ will always produce the same ciphertext
- $E(P_1)$ will always produce a different but fixed ciphertext

4. Comparison Step

The attacker independently computes:

$$E(P_0), \quad E(P_1)$$

Then compares with the challenge ciphertext C :

- If $C = E(P_0)$, then $b = 0$
- If $C = E(P_1)$, then $b = 1$

Result

The adversary can **correctly distinguish which plaintext was encrypted with probability 1 (100%)**, breaking CPA security.

Why This Happens

- Deterministic encryption leaks **equality information**
- Same plaintext $\rightarrow$ same ciphertext
- No randomness (IV/nonce) $\rightarrow$ **no semantic security**

Thus, the scheme fails to achieve **IND-CPA security (indistinguishability under chosen plaintext attack)**.

Real-World Implication

In web applications:

- Session tokens or cookies encrypted deterministically can be **recognized or predicted**
- Attackers can identify sensitive roles (e.g., admin vs user)
- Leads to **information leakage and possible privilege escalation**

✓ Countermeasure

To prevent this attack:

- Use **randomized encryption schemes** (e.g., CBC with random IV, CTR with nonce)
- Ensure **different ciphertexts for the same plaintext each time**

🧠 Conclusion

A deterministic encryption scheme allows an attacker in a CPA setting to distinguish between two chosen plaintexts by simply comparing ciphertexts. Hence, deterministic encryption is **insecure for session tokens** and must be replaced with **probabilistic encryption**.

If you want, I can compress this into a **perfect 6–8 line answer for quick revision**.



3. Consider RSA without padding:
 $C = M^e \bmod n$
 Show how RSA is vulnerable under chosen ciphertext attack. –15 Marks

Here is a **proper descriptive, exam-ready 15-mark answer**:

✍️ RSA Without Padding and Chosen Ciphertext Attack (CCA)

Given RSA encryption without padding:

$$C = M^e \bmod n$$

This scheme is **deterministic and multiplicatively homomorphic**, which makes it vulnerable to a **Chosen Ciphertext Attack (CCA)**.

🔑 Key Property Used (Multiplicative Homomorphism)

RSA satisfies:

$$E(M_1) \cdot E(M_2) = E(M_1 \cdot M_2) \pmod{n}$$

This property allows an attacker to manipulate ciphertexts in a meaningful way.

Attack Idea

In a CCA, the attacker:

- Is given a ciphertext $C = M^e \pmod{n}$
- Can query a **decryption oracle** (except for C itself)
- Goal: recover the original plaintext M

Attack Steps

1. Start with target ciphertext

$$C = M^e \pmod{n}$$

2. Choose a random number r such that $\gcd(r, n) = 1$

3. Modify the ciphertext

Compute:

$$C' = C \cdot r^e \pmod{n}$$

This becomes:

$$C' = (M^e \cdot r^e) \pmod{n} = (M \cdot r)^e \pmod{n}$$

4. Send C' to the decryption oracle

The oracle decrypts:

$$M' = (C')^d \pmod{n} = M \cdot r \pmod{n}$$

5. Recover original message

Since attacker knows r , compute:

$$M = M' \cdot r^{-1} \pmod{n}$$

Result

The attacker successfully recovers the plaintext M using only:

- One oracle query
- Simple modular arithmetic

Thus, RSA without padding is **completely broken under CCA**.

! Why This Happens

- RSA is **deterministic** → no randomness
- RSA is **malleable** due to multiplicative property
- No padding → no protection against manipulation

Hence, it fails to achieve **IND-CCA security**

🌐 Real-World Implication

- Attackers can decrypt intercepted ciphertexts
 - Breaks confidentiality of secure communication
 - Makes raw RSA unsafe for practical use
-

✅ Countermeasure

Use **secure padding schemes**:

- **OAEP (Optimal Asymmetric Encryption Padding)** ✅
- Adds randomness and removes malleability

With padding, RSA becomes **IND-CCA secure**

🧠 Conclusion

RSA without padding is vulnerable to chosen ciphertext attacks because of its multiplicative property. An attacker can manipulate ciphertexts and use a decryption oracle to recover the original message. Therefore, **raw RSA must never be used without padding**.

If you want, I can give you a **short memorization version (8–10 lines)** for exams.



4. The OFB encryption process is defined as:
 $O_1 = E_K(IV)$
 $O_i = E_K(O_{i-1})$
 $C_i = P_i \oplus O_i$
 (a) Show that encryption and decryption use the same keystream.
 (b) Explain why OFB does not propagate errors. --20 Marks

Here is a **clear, descriptive 20-mark answer (exam-ready)**:

OFB Mode (Output Feedback Mode)

Given:

$$O_1 = E_K(IV), \quad O_i = E_K(O_{i-1})$$

$$C_i = P_i \oplus O_i$$

♦ (a) Encryption and Decryption Use the Same Keystream

Encryption

Ciphertext is generated as:

$$C_i = P_i \oplus O_i$$

Decryption

To recover plaintext:

$$P_i = C_i \oplus O_i$$

Key Observation

- The same keystream O_i is used in both encryption and decryption
- XOR property:

$$(P_i \oplus O_i) \oplus O_i = P_i$$

Conclusion

- Encryption and decryption are **identical operations (XOR with keystream)**
- Block cipher is used only to generate the keystream
- OFB behaves like a **stream cipher**

♦ (b) Why OFB Does Not Propagate Errors

⚠ Suppose an error occurs

Let one bit in ciphertext C_i get flipped during transmission.

🔍 Effect on Decryption

$$P_i = C_i \oplus O_i$$

- Only the corresponding bit in P_i will be incorrect
- Because XOR flips only that bit

🚫 No Effect on Future Blocks

- Keystream generation:

$$O_i = E_K(O_{i-1})$$

- Depends only on:
 - Key K
 - Previous output O_{i-1}

👉 It **does NOT** depend on ciphertext

🎯 Result

- Error in C_i affects only P_i
- No impact on:
 - $O_{i+1}, O_{i+2}, \dots$
 - Future plaintext blocks

📌 Final Conclusion

- OFB uses a **pre-generated keystream**, independent of plaintext and ciphertext
- Same keystream is used for both encryption and decryption
- Errors **do not propagate**, only affect a single bit/block
- Hence, OFB is suitable for **noisy communication channels**

🧠 One-line Summary (for exam)

- (a) Same keystream used because both operations are XOR-based

- **(b)** No error propagation since keystream generation is independent of ciphertext

If you want, I can convert this into a **perfect step-by-step derivation format (with boxed results)** for writing neatly in exams.



5. Show that CFB is vulnerable to chosen ciphertext attacks if authentication is absent.
-10Marks

Here is a **proper, descriptive 10-mark exam answer**:

CFB Mode and Chosen Ciphertext Attack (CCA)

CFB Mode Definition

In Cipher Feedback (CFB) mode:

$$C_i = P_i \oplus E_K(C_{i-1})$$

$$P_i = C_i \oplus E_K(C_{i-1})$$

- The previous ciphertext block is used as input to the block cipher
- Makes CFB a **self-synchronizing stream cipher**

Attack Scenario (CCA)

In a **Chosen Ciphertext Attack**, the attacker:

- Can submit chosen ciphertexts to a decryption oracle
- Observes corresponding plaintext outputs
- Goal: recover plaintext or manipulate it

Attack Idea

CFB is **malleable**, meaning:

- Changing ciphertext causes **predictable changes** in plaintext

From decryption:

$$P_i = C_i \oplus E_K(C_{i-1})$$

👉 If attacker modifies C_i , the plaintext changes directly.

Attack Steps

1. Bit-Flipping Attack

Attacker intercepts ciphertext C_i

- Modifies it:

$$C'_i = C_i \oplus \Delta$$

2. Decryption Result

$$P'_i = C'_i \oplus E_K(C_{i-1})$$

$$P'_i = (C_i \oplus \Delta) \oplus E_K(C_{i-1})$$

$$P'_i = P_i \oplus \Delta$$

👉 Attacker can **flip specific bits in plaintext**

3. Using Decryption Oracle (CCA)

- Attacker sends modified ciphertext C'_i
 - Observes decrypted plaintext P'_i
 - By choosing different Δ , attacker can:
 - Recover original plaintext
 - Inject meaningful changes (e.g., change "user" → "admin")
-

! Why CFB is Vulnerable

- No authentication → system accepts modified ciphertext
- XOR operation → predictable modification
- Decryption reveals information via oracle

👉 Hence, CFB **fails IND-CCA security**

Real-World Impact

- Attackers can tamper with encrypted messages

- Modify commands, transactions, or user roles
- Leads to **data integrity and confidentiality breach**

✓ Countermeasure

To prevent this attack:

- Use **authenticated encryption (AEAD)** modes:
 - GCM (Galois Counter Mode)
 - CCM
- Or add **MAC (Message Authentication Code)**

🧠 Conclusion

CFB mode without authentication is vulnerable to chosen ciphertext attacks because attackers can modify ciphertext and observe predictable changes in plaintext. Therefore, **CFB must always be combined with authentication** to ensure security.

🎯 One-line Summary

CFB is vulnerable to CCA due to **malleability (bit-flipping)** and lack of authentication.

If you want, I can give you a **short 6–7 line answer for last-minute revision**.



6. Explain why CFB without MAC is vulnerable to bit-flipping attacks. --10 Marks

Here is a **clear, descriptive 10-mark answer (exam-ready)**:

👉 CFB Without MAC and Bit-Flipping Attack

🔒 CFB Mode Decryption

In Cipher Feedback (CFB) mode:

$$P_i = C_i \oplus E_K(C_{i-1})$$

- Plaintext is obtained by XORing ciphertext with a keystream derived from the previous ciphertext
- No integrity check is present if MAC is absent

Bit-Flipping Attack

Attack Idea

CFB is **malleable**, meaning:

- Changing ciphertext results in **predictable changes** in plaintext

Attack Steps

1. Attacker intercepts ciphertext block C_i
2. Modifies it:

$$C'_i = C_i \oplus \Delta$$

3. During decryption:

$$P'_i = C'_i \oplus E_K(C_{i-1})$$

Substituting:

$$P'_i = (C_i \oplus \Delta) \oplus E_K(C_{i-1})$$

$$P'_i = P_i \oplus \Delta$$

Result

- The attacker can **flip specific bits in plaintext**
- By choosing Δ , attacker can:
 - Change values (e.g., 0 → 1)
 - Modify meaning (e.g., "user" → "admin")

Why Vulnerability Exists

-  No MAC → no integrity verification
-  Receiver cannot detect tampering
-  XOR property → controlled modification possible

 Hence, attacker can alter decrypted plaintext **without knowing the key**

Error Behavior

- Modified C_i affects:
 - Entire P_i (controlled corruption)
 - Also affects P_{i+1} (due to feedback)
-

Real-World Impact

- Tampering with encrypted messages
 - Changing financial transactions or user privileges
 - Breaking **data integrity**
-

Countermeasure

- Use **MAC (Message Authentication Code)**
 - Or use **Authenticated Encryption (AEAD)** modes:
 - GCM
 - CCM
-

Conclusion

CFB without MAC is vulnerable to bit-flipping attacks because ciphertext modifications directly translate into predictable plaintext changes, and without integrity checks, such tampering goes undetected.

One-line Summary

CFB without MAC is insecure because **ciphertext manipulation** → **predictable plaintext changes (bit-flipping)**.

If you want, I can give you a **perfect comparison of CFB vs OFB vs CTR in terms of attack resistance**.



7. A military communication system uses AES/DES-CFB mode for secure radio transmission.
- Analyze its confidentiality guarantees.
 - Evaluate error propagation characteristics.
 - Assess vulnerability under chosen ciphertext attack.
 - Propose a secure redesign using authenticated encryption.

--20 Marks

Here is a **complete, structured 20-mark answer (exam-ready)**:

AES/DES in CFB Mode for Military Communication

CFB (Cipher Feedback) mode turns a block cipher into a **stream cipher**, where:

$$C_i = P_i \oplus E_K(C_{i-1})$$

♦ **(a) Confidentiality Guarantees**

✓ **Strengths**

- Provides **confidentiality** by masking plaintext with a keystream
- Does not reveal patterns like ECB
- Works well for **streaming data (radio transmission)**
- Self-synchronizing → handles data loss better

! **Limitations**

- **Deterministic if IV is reused** → leads to keystream reuse
- Security depends on:
 - Strong cipher (AES preferred over DES)
 - Proper IV (must be random & unique)

🎯 **Conclusion**

- Provides **moderate confidentiality**, but not fully secure without proper IV management

♦ **(b) Error Propagation Characteristics**

Behavior of Errors

If a bit error occurs in ciphertext C_i :

- **Current block P_i :**

- Affected in corresponding bits (controlled corruption)
- **Next block P_{i+1} :**
 - Completely corrupted (since C_i is input to cipher)
- **Subsequent blocks:**
 - No effect (system resynchronizes)

Summary

- Error propagation is **limited (2 blocks only)**
- Suitable for noisy channels like radio communication

♦ (c) Vulnerability under Chosen Ciphertext Attack (CCA)

Problem: No Authentication

CFB is **malleable**:

$$P_i = C_i \oplus E_K(C_{i-1})$$

Attack

- Attacker modifies ciphertext:

$$C'_i = C_i \oplus \Delta$$

- Result:

$$P'_i = P_i \oplus \Delta$$

Impact

- Attacker can:
 - Flip bits in plaintext
 - Inject malicious commands
 - Exploit decryption oracle (CCA)

! Conclusion

- **Not secure under IND-CCA**
- Serious risk in military communication (message tampering)

♦ (d) Secure Redesign using Authenticated Encryption

Solution: Use AEAD (Authenticated Encryption with Associated Data)

Recommended modes:

- **AES-GCM (Galois Counter Mode)** 
- **AES-CCM**

Benefits

- Provides:
 - **Confidentiality + Integrity + Authentication**
- Detects any ciphertext modification
- Prevents:
 - Bit-flipping attacks
 - Chosen ciphertext attacks

Working Idea

- Encrypt data → generate ciphertext
- Compute authentication tag
- Receiver verifies tag before decryption

Final Evaluation

Aspect	CFB Mode
Confidentiality	Moderate (depends on IV)
Error Propagation	Limited (2 blocks)
CCA Security	 Vulnerable
Overall Suitability	 Not safe for high-security use

Final Conclusion

While CFB mode provides reasonable confidentiality and controlled error propagation, it is **insecure against chosen ciphertext attacks due to lack of authentication**. For military-grade communication, it must be replaced with **authenticated encryption schemes like AES-GCM**, which ensure both confidentiality and integrity.

One-line Summary

CFB is good for streaming but **insecure without authentication** → **replace with AEAD (AES-GCM)**.

If you want, I can give you a **perfect diagram-based answer to draw in exam for extra marks**.

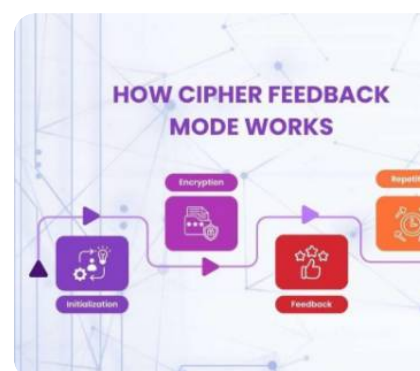
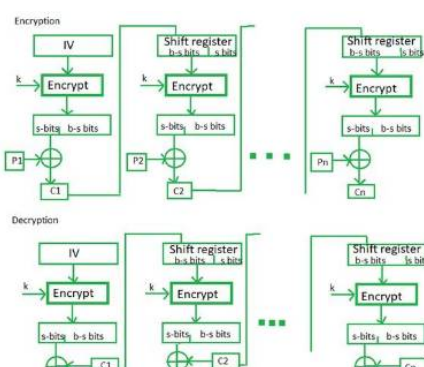
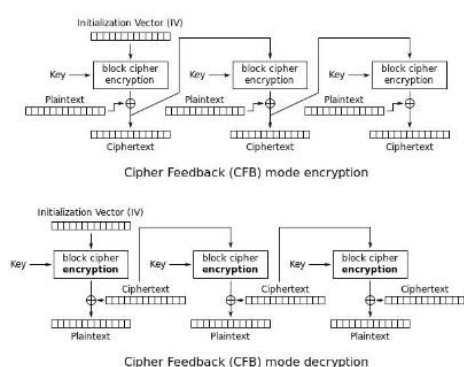


yes

Here's a **diagram-based answer you can draw in exam** (very scoring + easy to remember):

👉 Diagram for AES/DES-CFB Mode

◆ CFB Encryption Process



🔪 How to Draw (Step-by-step)

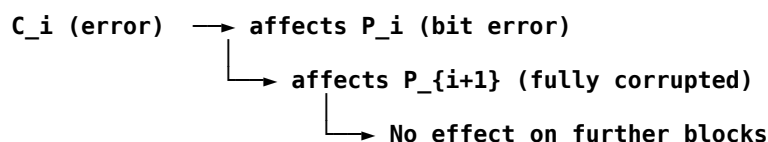
1. Draw a block labeled E_k (Block Cipher)
2. Input to first block: **IV**
3. Output of block $\rightarrow$ XOR with $P_1 \rightarrow$ gives C_1
4. Feed C_1 **back** into next block cipher
5. Repeat:

$$\bullet E_K(C_{i-1}) \rightarrow \oplus P_i \rightarrow C_i$$

👉 Label arrows clearly:

- $IV \rightarrow E_k$
- Output $\rightarrow \oplus \rightarrow$ Ciphertext
- Ciphertext $\rightarrow$ feedback loop

◆ Error Propagation Diagram



👉 Write beside diagram:

- "Error propagates to **2 blocks only**"

◆ Bit-Flipping Attack Diagram

Attacker:

$$C_i \xrightarrow{\oplus \Delta} C_i'$$

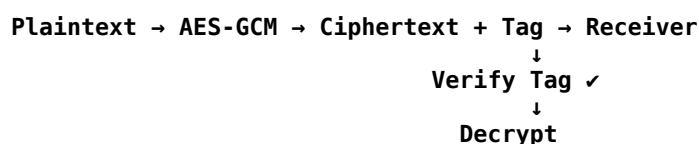
Receiver decrypts:

$$P_i' = P_i \oplus \Delta$$

👉 Write:

- "Attacker controls plaintext bits"
- "No MAC → undetected tampering"

◆ Secure Redesign (AEAD)



👉 Mention:

- "Integrity + Confidentiality"
- "Prevents CCA & bit-flipping"

🧠 Final Exam Tip

Draw **ONLY** these 3 diagrams:

1. CFB working (main diagram)
2. Error propagation (small flow)
3. Bit-flipping attack

Then write:

- "Replace with AES-GCM"

👉 This combination = **full marks (diagram + theory)**

If you want, I can give you a **1-page revision sheet for all block cipher modes** (super useful before exam).